```
epochs = 75
lr = 0.0001
num_iter = 3
1
losses = [ [] for i in range(size[0])]
2
replay_size = 50
3
replay = deque(maxlen=replay_size)
4
batch_size = 10
5
gamma = 0.9
6
losses = [[] for i in range(J)]

for i in range(epochs):
    act_means = torch.zeros((J,2))
7
    q_next = torch.zeros(J)
8
    for m in range(num_iter):
9
        for j in range(J):
10
            action_mean = mean_action(grid_,j).detach()
            act_means[j] = action_mean.clone()
            qvals =
qfunc(action_mean.detach(),params[0],layers=layers)
            action = softmax_policy(qvals.detach(),temp=0.5)
            grid__[get_coords(grid_,j)] = action
            q_next[j] = torch.max(qvals).detach()
        grid_.data = grid__.data
    grid.data = grid_.data
    actions = torch.stack([get_substate(a.item()) for a in
grid.flatten()])
    rewards = torch.stack([get_reward_2d(actions[j],act_means[j])
for j in range(J)])
    exp = (actions,rewards,act_means,q_next)
11
    replay.append(exp)
    shuffle(replay)
    if len(replay) > batch_size:
1@
        ids =
np.random.randint(low=0,high=len(replay),size=batch_size)   13
        exps = [replay[idx] for idx in ids]
        for j in range(J):
            jacts = torch.stack([ex[0][j] for ex in exps]).detach()
            jrewards = torch.stack([ex[1][j] for ex in
```